



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE INGENIERÍA
Año 2025 - 2^{er} Cuatrimestre

TALLER DE PROGRAMACION - DOCUMENTACION TECNICA

CORRECTORAS: ABRIL DIAZ MIGUEZ - NATHALIA ENCINOZA

GRUPO 7

INTEGRANTES: NICOLAS CARDONE - 111148

ALEXIS DANIEL GUTTLEIN GAREIS - 104431

VALENTIN PROVOST - 111537

FECHA: 5/12/2025

Índice

1. Introducción	3
2. Diagramas	3
3. Protocolo	6
4. Formato de los archivos	7

1. Introducción

El presente documento describe cómo fue modelado y desarrollado el Need For Speed 2D, realizado como Trabajo Práctico Final de la materia Taller de Programación I. El objetivo del juego es que varios jugadores compitan en distintos circuitos, manejando sus autos elegidos. El servidor se encarga de simular la carrera de forma autoritativa (física, colisiones, vueltas, checkpoints, estado de cada auto) y los clientes se encargan de capturar la entrada del usuario y dibujar en pantalla el estado más reciente de la partida. Para comprender en profundidad la implementación se asume que el lector posee conocimientos básicos de:

- Programación orientada a objetos en C++.
- Programación concurrente y uso de threads.
- Comunicación mediante sockets.
- Uso de SDL2/SDL2pp para la parte gráfica y de Box2D para la física.

2. Diagramas

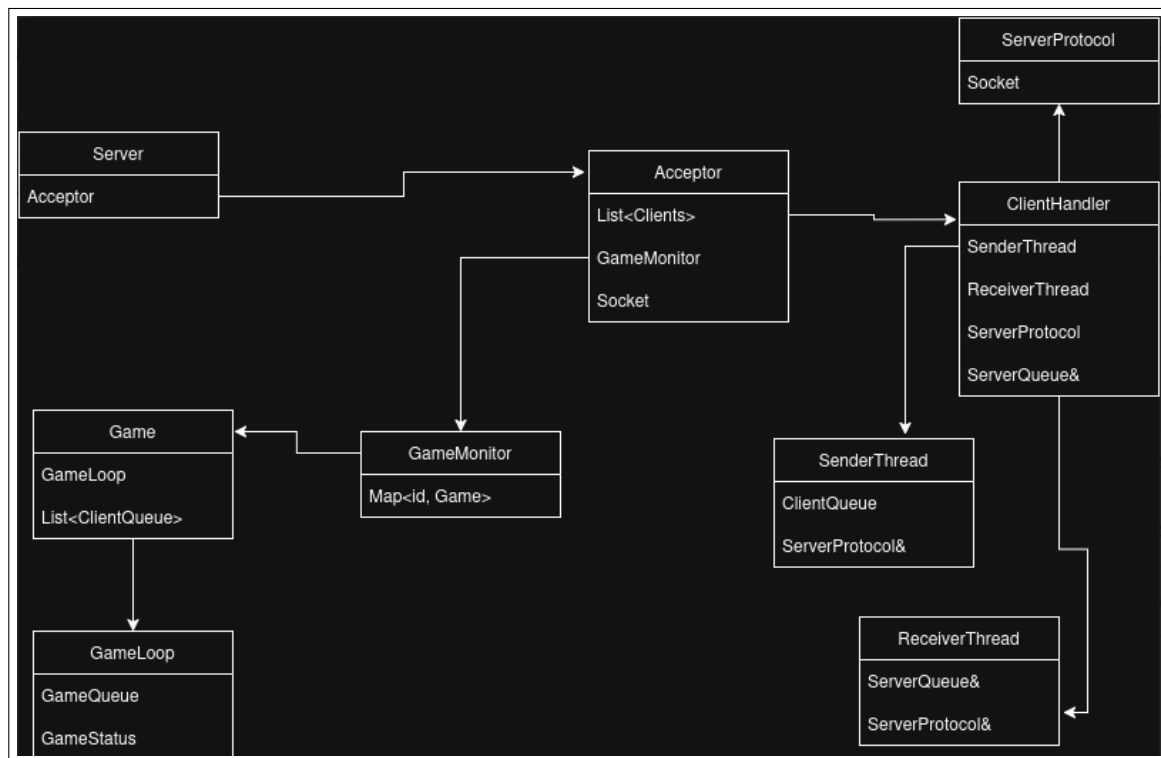


Figura 1: El siguiente diagrama de clases representa la relacion entre las clases y los threads que utilizados

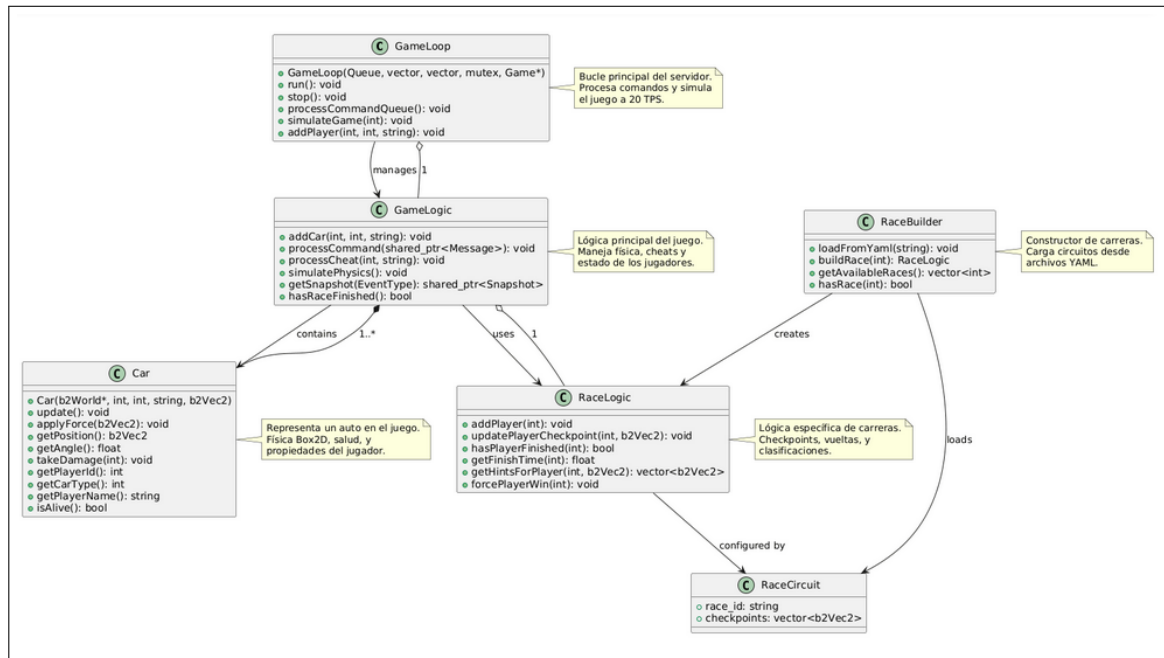


Figura 2: El siguiente diagrama de clases representa la relacion entre las clases que componen la lógica del lado del servidor

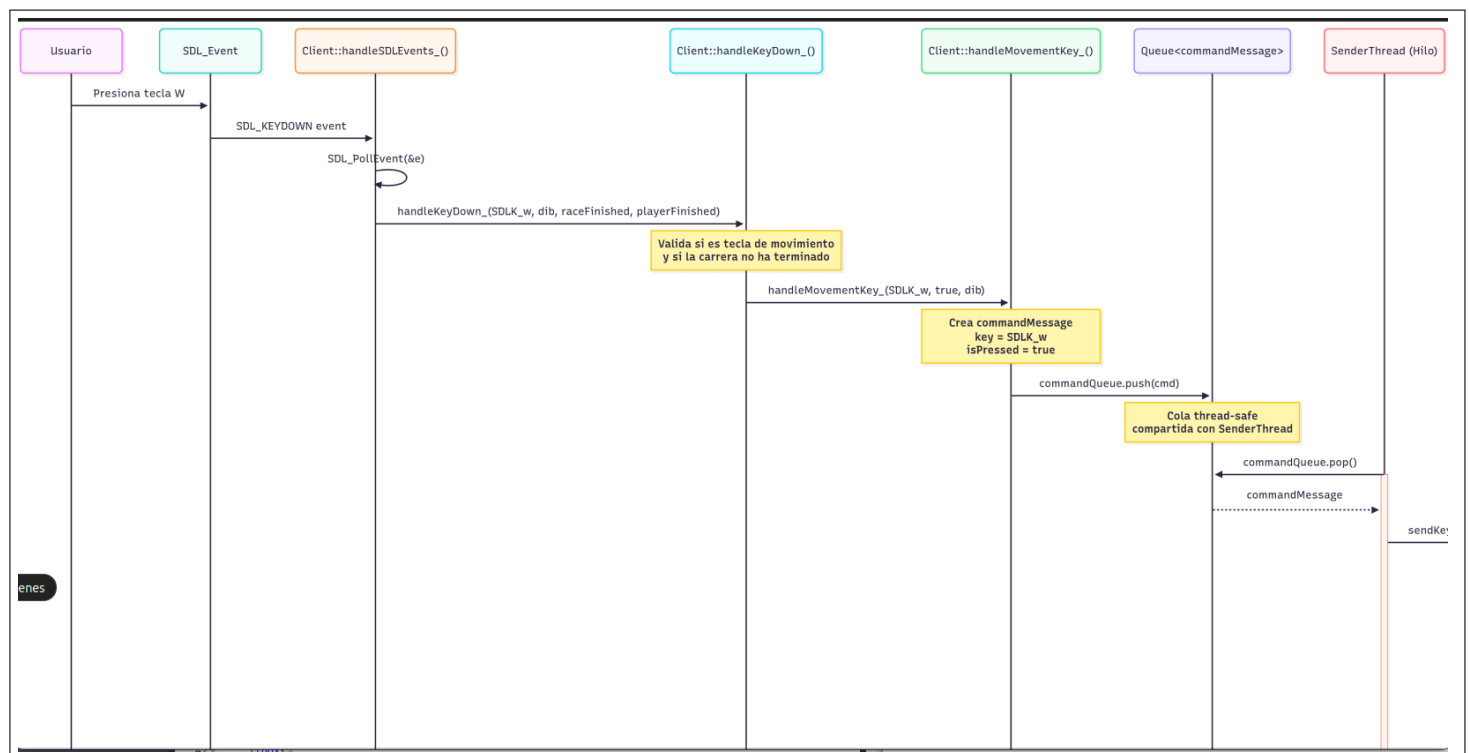


Figura 3: Diagrama de secuencia: procesamiento de la tecla

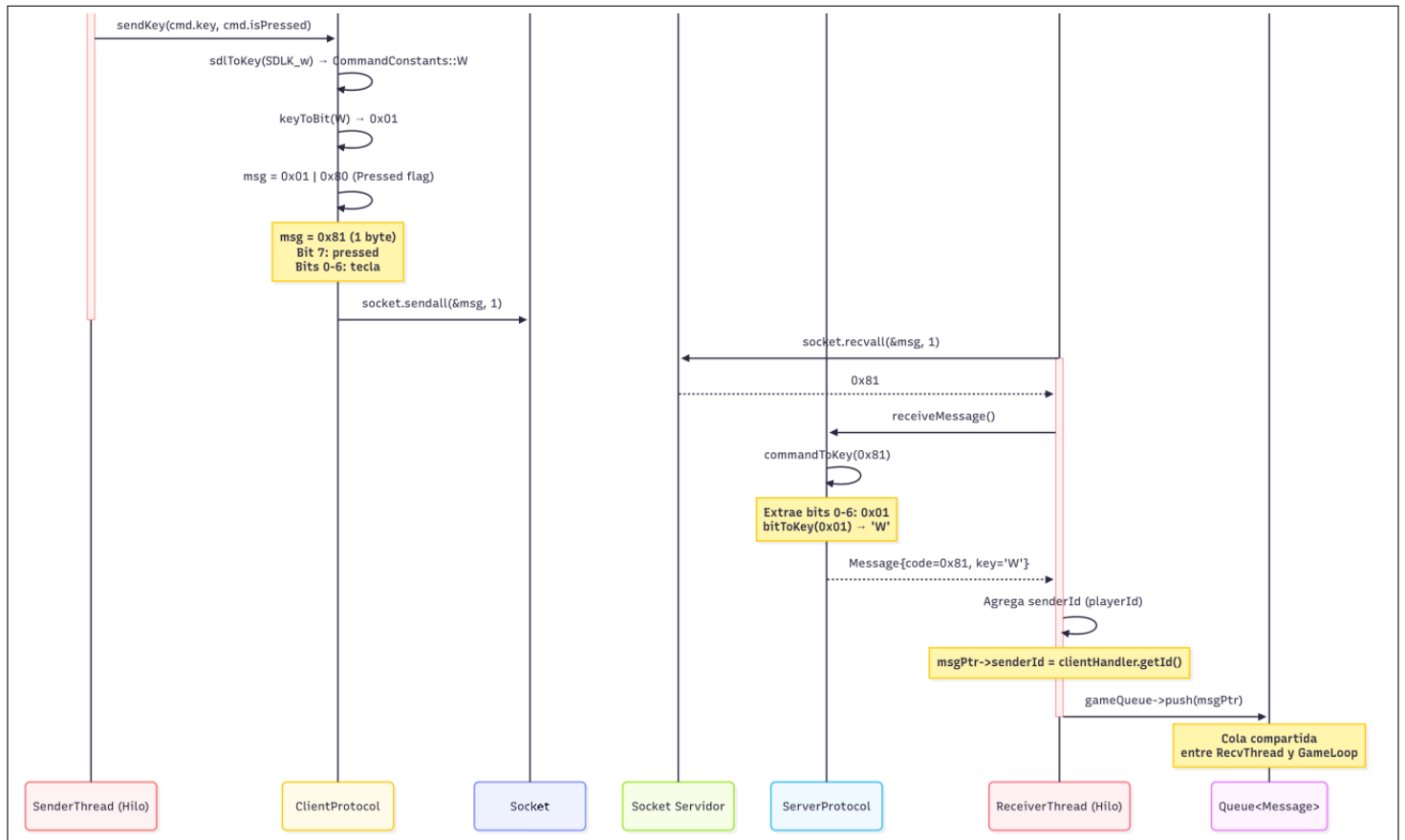


Figura 4: Diagrama de secuencia: procesamiento de la tecla

En este diagrama se visualiza el flujo desde que se toca una tecla el usuario, hasta que se la envía al servidor.

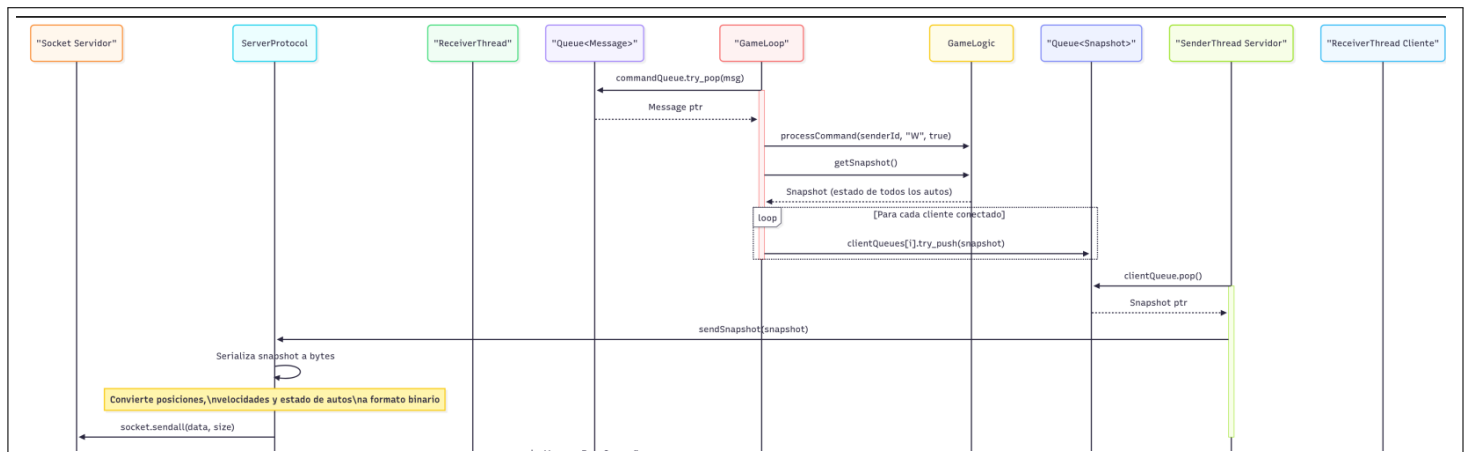


Figura 5: Diagrama de secuencia: procesamiento del snapshot

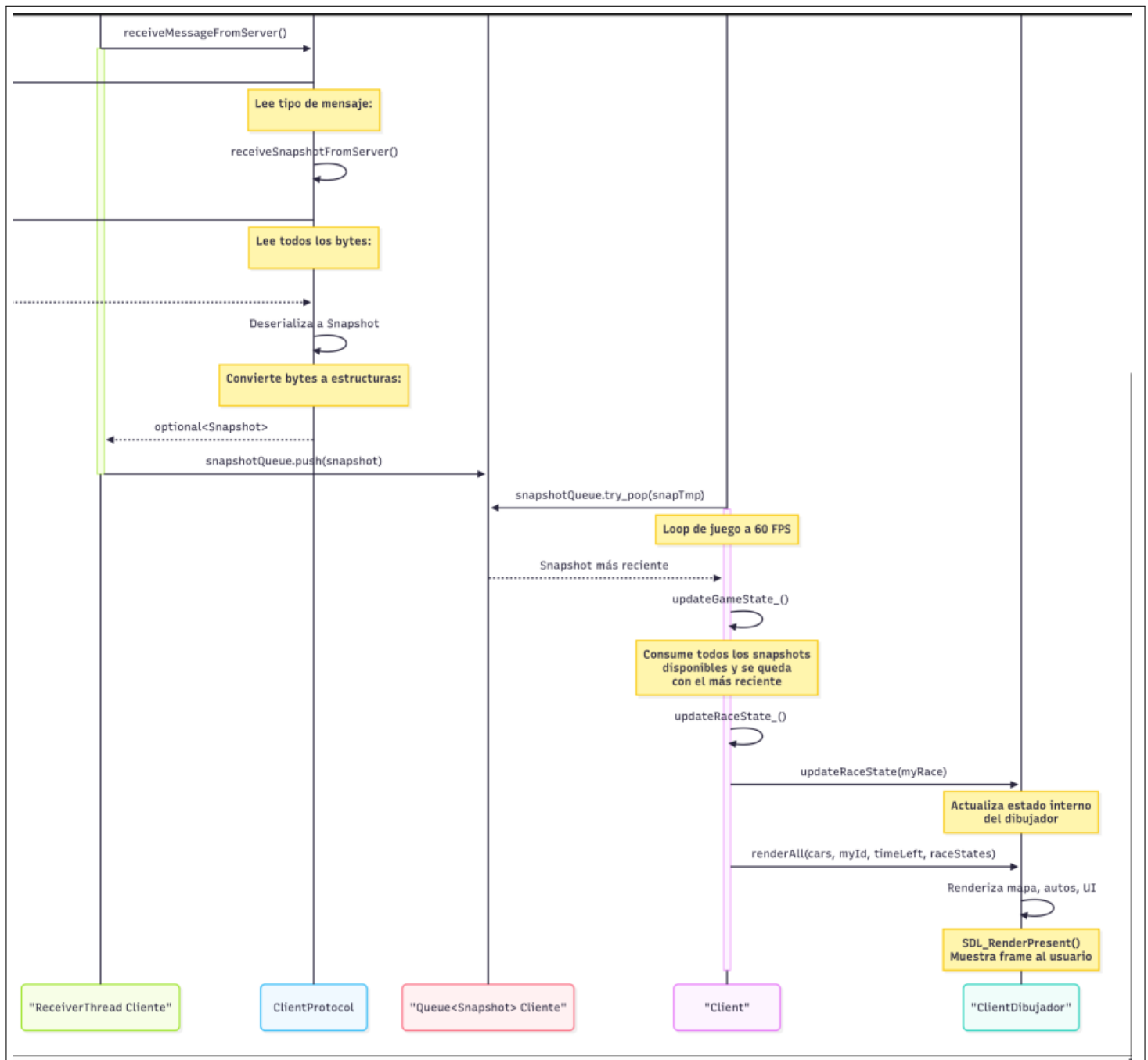


Figura 6: Diagrama de secuencia: procesamiento del snapshot

En este diagrama se visualiza el flujo desde que el servidor recibe un comando del cliente, lo procesa y como se envia el snapshot al cliente grafico para que lo muestre.

3. Protocolo

El protocolo utilizado para la comunicación entre el cliente y el servidor es binario, diseñado específicamente para el juego.

La comunicación se realiza mediante serialización y deserialización de estructuras en formato binario, enviando y recibiendo secuencias de bytes que representan tanto entradas del jugador como el estado actualizado de la partida.

Lado del cliente: El cliente serializa las teclas accionadas por el jugador (acelerar, frenar, girar, etc.) y las envía al servidor en forma de mensajes binarios compactos.

Lado del servidor: El servidor procesa estas entradas, actualiza el estado del juego y envía al cliente un snapshot del estado completo relevante para ese jugador. Este snapshot se envía también como un mensaje binario serializado, conteniendo información como la posición de los autos, velocidades, colisiones, elementos del mapa, etc.

4. Formato de los archivos

Los archivos del proyecto se basan principalmente en formato YAML. Cada archivo define datos específicos del juego de forma externa al código.

El archivo `game settings.yaml` contiene parámetros generales como tiempo máximo de carrera o cantidad de jugadores. Su formato es clave-valor y nuestro settings Loader lo parsea al inicio para cargarlo en memoria y exponerlo al resto del juego.

El archivo `racess.yaml` contiene los checkpoints de cada carrera cargada

También utilizamos mapas generados en Tiled, cuyo formato exportado (JSON) nos ayuda a setear los colisionables en el mundo de box2d. Están dados en el archivo: `mapa1 nfs.yaml` y se parsean en `map set objects` y `yamlLoader`.